



Jour 2 Développement Web

De Statique à Dynamique

Les Limites du Statique

Leçon du Jour 1 : La Base

On a vu comment utiliser HTML pour structurer le contenu et CSS pour sa présentation visuelle. C'est la fondation essentielle de toute page web. Cependant, un site purement statique présente des limitations importantes dans le monde actuel :

- Il **ne s'adapte pas** aux différents supports (PC, tablette, smartphone). Votre CV d'hier pourrait être illisible sur mobile.
- Il **n'interagit pas** avec l'utilisateur. Impossible d'avoir des formulaires fonctionnels, des boutons cliquables ou une expérience personnalisée.
- Il affiche toujours **le même contenu**. Les données ne sont pas mises à jour (météo, tweets récents, données GitHub, etc.).



Aujourd'hui, nous allons dépasser ces limites. Nous allons rendre vos sites **responsive**, ajouter de l' **interactivité** avec JavaScript, et simuler la connexion à des **données dynamiques** via des APIs.

L'Anatomie d'un Site Web Moderne : L'Analogie du Restaurant

Pour bien comprendre comment un site web moderne fonctionne, imaginons-le comme un restaurant bien huilé.



HTML : La Salle à Manger

C'est la structure visible, les tables, les chaises, le menu. Sans elle, le restaurant n'existe pas.



JavaScript : Les Serveurs

Ils prennent les commandes, apportent les plats, interagissent avec les clients. Ils donnent vie à la salle.



Base de Données : Le Garde-Manger

Tous les ingrédients sont stockés ici : recettes, stocks, préférences des clients. C'est la mémoire du restaurant.



CSS : L'Ambiance et la Décoration

Les couleurs des murs, l'éclairage, la disposition des tables. Cela rend la salle agréable et accueillante.



Back-end : La Cuisine

C'est là que la magie opère, où les plats sont préparés selon les recettes (logique métier). Les clients ne la voient pas directement.



API : L'Interface Cuisine-Salle

Le serveur (JavaScript) utilise l'API pour communiquer avec la cuisine (Back-end) et le garde-manger (Base de Données). C'est le pont entre ce que voit le client et ce qui se passe en coulisses.

Sans l'harmonie de ces éléments, le restaurant ne fonctionnerait pas efficacement. De même, un site web moderne nécessite ces trois piliers : le **Front-end** (HTML, CSS, JS), le **Back-end** et la **Base de Données**, tous connectés via des **APIs**.

L'Évolution de Votre CV : Statique vs. Dynamique

Pour illustrer ce changement, reprenons votre CV codé hier et imaginons son évolution.

CV Statique (Jour 1)



- Même texte et même mise en page pour tous, sans possibilité de personnalisation.
- Absence totale d'interaction (boutons non cliquables, etc.).
- Affichage non optimisé sur mobile, nécessitant un défilement horizontal.
- Mise à jour manuelle de chaque information.

CV Dynamique (Jour 2)



- **Responsive** : lisible et bien organisé sur smartphone, tablette et ordinateur.
- **Interactif** : Intégration de fonctionnalités comme un bouton "mode clair/sombre" ou un formulaire de contact fonctionnel.
- **Connecté** : Possibilité de charger dynamiquement une liste de projets depuis une API GitHub, par exemple.
- Mise à jour centralisée des données via une base de données, reflétée instantanément.

⊗ Ne tombez pas dans le piège ! Un site juste "joli" mais statique n'est pas compétitif. Les utilisateurs attendent fluidité, adaptabilité mobile et données à jour. Évoluez !

Exercice pratique (15 min)

1. **Binôme** : montrez votre CV du Jour 1 à votre voisin.
2. Chacun doit identifier **3 défauts concrets** :
 - Problème d'affichage sur petit écran ?
 - Design peu lisible ?
 - Manque d'interactivité ?
3. Ensemble, proposez **3 pistes d'amélioration** :

CSS Avancé : Architecture & Layouts Professionnels

Jusqu'ici, vous avez utilisé le CSS pour changer des couleurs, des polices et ajouter un peu de style. Mais dans un vrai projet web, le CSS doit résoudre **deux grands défis** :

- Architecture** → Comment organiser et nommer vos styles pour qu'ils restent compréhensibles, même dans un projet de 2000 lignes ?
- Layouts (mises en page)** → Comment placer correctement vos éléments pour obtenir des pages modernes, adaptatives et lisibles ?

Aujourd'hui, nous allons aborder :

- la **spécificité CSS** (qui gagne quand plusieurs styles s'appliquent),
- les **conventions de nommage** (comme le BEM),
- deux systèmes puissants de mise en page modernes : **Flexbox** et **Grid**.

1. La spécificité CSS

Quand plusieurs règles CSS ciblent le même élément, c'est une question de **priorité**.

👉 Règle générale :

- Un sélecteur **inline** (dans le HTML) est le plus fort.
- Ensuite viennent les **IDs**.
- Puis les **classes**.
- Enfin les **balises** (p, h1, etc.).

Exemple

```
p { color: blue; }      /* priorité faible */
.texte { color: red; } /* plus fort */
#important { color: green; } /* écrase tout */
```

👉 Résultat : le texte sera **vert**, car l'ID gagne.

⚠ **Piège** : utiliser `!important` partout → ça casse toute logique et rend le code incontrôlable.

2. Conventions de nommage (BEM)

Dans les gros projets, on a vite des centaines de classes. Sans règles de nommage, c'est le chaos.

La convention **BEM (Block Element Modifier)** est une bonne pratique répandue :

- Block** : le composant principal.
- Element** : une partie du composant.
- Modifier** : une variante.

Exemple

```
.card {}           /* Block = carte */
.card_title {}    /* Element = titre de la carte */
.card--blue {}    /* Modifier = carte en bleu */
```

💡 **Avantage** : le code reste clair et facile à maintenir.

3. Flexbox : alignements modernes

Flexbox est un système pour organiser les éléments **en ligne ou en colonne**, avec des alignements puissants.

Exemple

```
.container {
  display: flex;
  justify-content: space-between; /* espace égal entre les éléments */
  align-items: center;           /* alignement vertical */
}
```

Cas d'usage

- Navigation horizontale.
- Cartes de profil alignées.
- Centrage vertical parfait.

⚠ Avant Flexbox, on utilisait des "hacks" (comme `float` ou `table`). Flexbox simplifie tout.

4. CSS Grid : mise en page en 2D

Flexbox gère bien **une seule dimension** (ligne ou colonne).

Grid gère **deux dimensions** (lignes + colonnes).

Exemple

```
.grid {
  display: grid;
  grid-template-columns: 1fr 2fr 1fr; /* 3 colonnes */
  grid-template-areas:
    "sidebar main aside";
  gap: 20px;
}
.sidebar { grid-area: sidebar; }
.main { grid-area: main; }
.aside { grid-area: aside; }
```

Cas d'usage

- Layout complet de page (sidebar, contenu, colonne droite).
- Magazines, dashboards.

💡 **Bonnes pratiques** :

- Flexbox** = composants.
- Grid** = structure globale de page.

Comparaison rapide

Système	Dimension	Cas d'usage	Exemple pratique
Flexbox	1D	Menus, galeries, boutons	Centrer un bloc
Grid	2D	Pages complètes, tableaux	Layout 3 colonnes

Exercice pratique (25 min)

Objectif

Transformer le CV du Jour 1 en **layout 3 colonnes** avec Grid.

Étapes guidées

- Créez un container `.cv-grid` avec `display: grid;`
- Définissez 3 colonnes :

```
grid-template-columns: 1fr 2fr 1fr;
```

- Placez vos sections dans chaque colonne :
 - Sidebar gauche : photo + contacts.
 - Colonne centrale : profil + formation + expériences.
 - Colonne droite : compétences.
- Ajoutez un `gap: 20px;` pour aérer.

Variante (bonus)

→ Essayez aussi la version **Flexbox** pour comparer les deux approches.

Responsive Design & Variables CSS

Aujourd'hui, **plus de 60% de la navigation web se fait sur mobile**.
Si votre site n'est pas lisible sur un écran de smartphone → vous perdez vos utilisateurs.

Un **site responsive** est un site qui **s'adapte automatiquement** à la taille de l'écran.
Il ne suffit pas de "rétrécir la page" : il faut **penser différemment la mise en page**, et utiliser des outils **CSS** modernes.

Deux points clés pour réussir un design adaptatif :

1. **Unités relatives** (pour que les tailles soient flexibles).
2. **Media Queries** (pour changer le style selon la largeur d'écran).

Et pour garder une identité visuelle claire et facilement modifiable → on introduit les **variables CSS**.

1. Les unités CSS : absolues vs relatives

Unités absolues

- px (pixels) → toujours la même taille.
- ✗ Problème : sur un petit écran, tout devient trop petit.

Unités relatives

- % → relatif à la taille du parent.
- em → relatif à la taille de la police du parent.
- rem → relatif à la police racine du document.
- vh / vw → relatif à la hauteur ou largeur de l'écran.

👉 Exemple :

```
h1 { font-size: 2.5rem; } /* proportionnel à la racine */
.hero { height: 80vh; } /* occupe 80% de la hauteur écran */
```

Analogie

Imaginez un vêtement :

- Taille fixe = "S" ou "M" (comme px).
- Taille ajustable = "s'adapte à votre morphologie" (comme rem/vh/vw).

2. Les variables CSS

Les variables CSS permettent de **centraliser les styles importants** (couleurs, polices, espacements).
C'est comme une **palette partagée** → changer une seule valeur met tout le site à jour.

Exemple

```
:root {
  --primary-color: #3498db;
  --secondary-color: #2c3e50;
  --font-size-base: 16px;
}

body {
  font-size: var(--font-size-base);
  color: var(--secondary-color);
}

h1 {
  color: var(--primary-color);
}
```

👉 Avantage :

- Cohérence visuelle.
- Facile à maintenir.
- Utile pour implémenter un **mode clair/sombre**.

3. Les Media Queries

Une **Media Query** est une règle CSS conditionnelle :
"Applique ce style si la largeur de l'écran est ≤ ou ≥ à telle valeur."

Exemple simple

```
/* Desktop (par défaut) */
.grid { grid-template-columns: 1fr 2fr 1fr; }

/* Mobile */
@media (max-width: 768px) {
  .grid { grid-template-columns: 1fr; }
}
```

Approches possibles

- **Desktop First** → on réduit progressivement.
- **Mobile First** → on part du plus petit écran et on élargit.
- 👉 En 2025, la méthode **Mobile First** est recommandée.

⚠️ Pièges fréquents

1. Utiliser **trop de media queries** (une pour chaque taille d'écran).
→ Mauvaise pratique, code ingérable.
✅ Solution : se limiter à 2 ou 3 breakpoints.
2. Utiliser uniquement des **unités fixes (px)**.
→ Résultat : site qui ne s'adapte pas.
✅ Solution : préférer %, rem, vh.
3. Ne pas tester son site sur plusieurs écrans.
✅ Solution : utiliser **Chrome DevTools** (mode mobile/tablette).

✍️ Exercice pratique (25 min)

Objectif

Rendre votre CV **adaptatif** avec variables CSS et media queries.

Étapes guidées

1. Définir des variables CSS globales dans `:root` :
 - Couleur principale
 - Couleur secondaire
 - Police principale
2. Implémenter deux thèmes :
 - Clair (fond blanc, texte noir)
 - Sombre (fond noir, texte clair)
→ Astuce : stocker les variantes dans `.theme-dark` et `.theme-light`.
3. Ajouter **2 breakpoints** :
 - Desktop (≥ 1024px) → CV en 3 colonnes.
 - Mobile (≤ 768px) → CV en 1 seule colonne.
4. Tester le résultat avec Chrome DevTools.

Variante (bonus)

→ Créez une **media query pour tablette** (entre 768px et 1024px) → CV en 2 colonnes.

Responsive Design et Variables CSS : L'Art de l'Adaptation

Avec plus de 60% de la navigation web sur mobile, un site qui ne s'adapte pas est un site qui perd des utilisateurs. Le Responsive Design n'est pas une option, c'est une nécessité.

Unités CSS : Relatives vs. Absolues



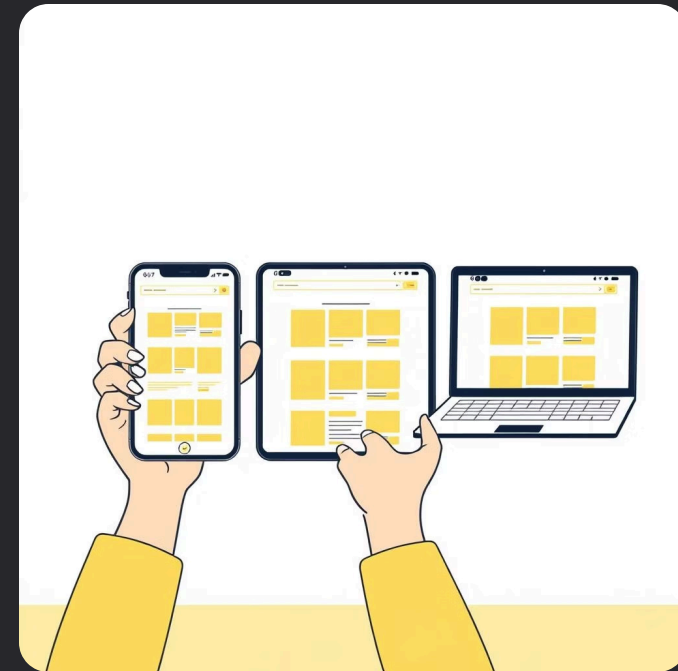
- **Absolues** (px) : Taille fixe, inadaptée aux écrans variés. Imaginez un vêtement à taille fixe.
- **Relatives** (% , em, rem, vh, vw) : S'ajustent à la taille du parent ou de l'écran. C'est comme un vêtement ajustable. Elles sont **essentiels** pour un design fluide.

Variables CSS : La Palette Globale

Elles centralisent vos styles (couleurs, polices, espacements). Modifier une variable met à jour tout votre site, assurant cohérence visuelle et facilité de maintenance.

```
:root {
  --primary-color: #3498db;
  --font-size-base: 16px;
}
body {
  font-size: var(--font-size-base);
  color: var(--primary-color);
}
```

Media Queries : Le Style Conditionnel



C'est la magie derrière le responsive. Une Media Query applique des styles spécifiques en fonction des caractéristiques de l'écran (ex. largeur).

```
/* Desktop (par défaut) */
.grid { grid-template-columns: 1fr 2fr 1fr; }

/* Mobile */
@media (max-width: 768px) {
  .grid { grid-template-columns: 1fr; }
}
```

Mobile First est l'approche recommandée en 2025 : concevez d'abord pour les petits écrans, puis enrichissez pour les plus grands.

En combinant unités relatives, variables CSS et Media Queries, vous construirez des sites qui s'affichent parfaitement sur n'importe quel appareil, offrant une expérience utilisateur **irréprochable**.

JavaScript : DOM et Événements - L'Interactivité au Cœur du Web

Après la structure (HTML) et la présentation (CSS), JavaScript (JS) est le moteur qui donne vie à vos pages web.

JavaScript est un langage de programmation qui permet de :

- 1. **Modifier dynamiquement** le contenu et le style d'une page.
- 2. **Réagir aux actions de l'utilisateur** (clics, saisies, défilement, etc.).
- 3. **Communiquer avec des serveurs** pour récupérer ou envoyer des données (APIs).

Le DOM : La Représentation de Votre Page

Le **DOM (Document Object Model)** est une **représentation en mémoire** de votre page HTML.

- Chaque balise HTML devient un **noeud** dans l'arbre du DOM.
- JavaScript peut accéder à ces noeuds pour les **modifier, supprimer, créer**.

```
/* Sélectionner un élément */
const monParagraphe = document.getElementById("monId");
const mesBoutons = document.querySelectorAll(".bouton");

/* Modifier le contenu */
monParagraphe.innerText = "Nouveau texte !";

/* Modifier le style */
monParagraphe.style.color = "blue";
monParagraphe.classList.add("actif");
```

Les Événements : Réogir aux Actions

Les événements sont des actions qui se produisent dans le navigateur, comme un clic de souris, une saisie au clavier, ou le chargement d'une page. JavaScript permet d'**écouter** ces événements et de **réagir** en exécutant du code.

```
const monBouton = document.querySelector("#monBouton");

/* Écouter un clic sur le bouton */
monBouton.addEventListener("click", function() {
  alert("Bouton cliqué !");
});

/* Bonnes pratiques */
// 1. Séparer HTML, CSS, JS
// 2. addEventListener > onclick="..."
// 3. Noms de variables clairs
```

 Ne mettez pas vos balises `<script>` dans le `<head>` sans l'attribut `defer`, ou mieux encore, placez-les à la fin du `<body>` pour que le DOM soit entièrement chargé avant que JavaScript n'interagisse avec.

2. Sélectionner des éléments dans le DOM

Pour manipuler un élément, il faut d'abord le **sélectionner**.

Principales méthodes :

```
document.getElementById("id"); // par id
document.querySelector(".class"); // par sélecteur CSS
document.querySelectorAll("p"); // tous les <p>
```

Exemple pratique

```
<p id="demo">Texte initial</p>
<script>
  const paragraphe = document.getElementById("demo");
  paragraphe.innerText = "Texte modifié !";
</script>
```

3. Modifier le contenu et le style

Une fois sélectionné, on peut changer :

- **Le texte** → `.innerText` ou `.innerHTML`
- **Le style** → `.style.property`
- **Les classes** → `.classList.add()` / `.classList.remove()`

Exemple

```
paragraphe.style.color = "red"; // change la couleur en rouge
paragraphe.classList.add("important"); // ajoute une classe
```

4. Réagir aux événements

Un **événement** est une action de l'utilisateur : clic, saisie clavier, défilement..

👉 On peut "écouter" ces événements avec `addEventListener`.

Exemple

```
<button id="btn">Cliquez-moi</button>
<p id="message"></p>

<script>
  const bouton = document.getElementById("btn");
  bouton.addEventListener("click", () => {
    document.getElementById("message").innerText = "Bouton cliqué !";
  });
</script>
```

👉 À chaque clic → le texte s'affiche.

⚠ Pièges fréquents

1. **Mettre `<script>` dans le `<head>` sans attendre le chargement**
→ JS essaye de manipuler des éléments qui n'existent pas encore.
✅ Solution : placer le script en bas du `<body>`.
2. **Confusion entre `innerText` et `innerHTML`**
 - `innerText` → change uniquement le texte.
 - `innerHTML` → insère du HTML (plus puissant mais dangereux si mal utilisé).
3. **Mélanger HTML et JS** (ex. `onclick="..."` dans le HTML)
→ acceptable pour débiter, mais mauvaise pratique en projet réel.

💡 Bonnes pratiques

- Toujours séparer **HTML** (structure), **CSS** (style), **JS** (comportement).
- Utiliser `addEventListener` plutôt que `onclick` → plus propre et modulaire.
- Donner des noms de variables clairs (éviter x, y, z).

📝 Exercice pratique (30 min)

Objectif

Ajouter de l'interactivité au CV (ou à une page test).

Étapes guidées

1. Créer un bouton "Changer la couleur du fond".
 - Au clic → le `body` passe en bleu.
2. Créer un bouton "Ajouter une compétence".
 - Au clic → ajouter `<li>Nouvelle compétence</li>` dans la liste de compétences.
3. Créer un bouton "Afficher la date du jour".
 - Au clic → afficher la date actuelle dans le footer.

Variante (bonus)

→ Ajouter un **toggle clair/sombre** avec `classList.toggle("dark-mode")`.

Formulaires, Validation et LocalStorage : Interagir avec l'Utilisateur

UUn site moderne ne se contente pas d'afficher du texte : il doit **collecter des données** auprès des utilisateurs.

👉 Exemple : formulaire de contact, formulaire d'inscription, formulaire de paiement.

Problème

- Les utilisateurs peuvent **oublier de remplir un champ**.
- Ou entrer une donnée **incorrecte** (ex. email sans @).

Il est donc indispensable de mettre en place une **validation côté client** avec **JavaScript**, avant même d'envoyer les données au serveur.

De plus, pour améliorer l'expérience utilisateur, on peut **sauvegarder certaines informations** directement dans le navigateur avec le **LocalStorage**.

1. Structure d'un formulaire HTML

Un formulaire contient :

- Des champs de saisie (<input>, <textarea>).
- Un bouton de soumission (<button type="submit">).

Exemple simple

```
<form id="contactForm">
  <label for="name">Nom :</label>
  <input type="text" id="name" required>

  <label for="email">Email :</label>
  <input type="email" id="email" required>

  <label for="message">Message :</label>
  <textarea id="message"></textarea>

  <button type="submit">Envoyer</button>
</form>
```

👉 Ici, l'attribut **required** empêche l'envoi si le champ est vide (validation HTML de base).

Validation de Formulaires avec JS

Les formulaires HTML incluent une validation de base (ex. **required**), mais JavaScript permet des contrôles plus fins. Il bloque l'envoi du formulaire si les données ne respectent pas les critères définis (ex. email avec un '@').

```
document.getElementById("contactForm").addEventListener("submit", function(e) {
  e.preventDefault(); // empêche l'envoi automatique

  const name = document.getElementById("name").value;
  const email = document.getElementById("email").value;

  if (name === "") {
    alert("Le nom est obligatoire !");
    return;
  }

  if (!email.includes("@")) {
    alert("Email invalide !");
    return;
  }

  alert("Formulaire envoyé avec succès");
```

```
    ✓");
  });
```

⊗

La validation JS est un confort pour l'utilisateur, mais ne remplace JAMAIS la validation côté serveur pour des raisons de sécurité.

⚠️ Pièges fréquents

1. **Se fier uniquement au JavaScript pour la validation**
→ Un utilisateur malintentionné peut désactiver JS.
✅ Solution : il faut aussi valider côté serveur.
2. **Stocker des données sensibles dans LocalStorage**
→ Exemple : mots de passe, numéros de carte bancaire.
✅ Interdit, car LocalStorage est facilement accessible.
3. **Oublier preventDefault()**
→ Sans ça, le formulaire se soumet immédiatement et recharge la page.

💡 Bonnes pratiques

- Toujours combiner **validation HTML de base** + **validation JS** + **validation serveur**.
- Ne sauvegarder dans LocalStorage que des infos **non sensibles** (préférences, prénoms, thèmes, etc.).
- Effacer LocalStorage quand ce n'est plus nécessaire.

📝 Exercice pratique (30 min)

Objectif

Ajouter un formulaire **Contact** au CV et améliorer l'expérience utilisateur avec LocalStorage.

Étapes guidées

1. Créer un formulaire avec :
 - Champ Nom (text)
 - Champ Email (email)
 - Champ Message (textarea)
 - Bouton Envoyer
2. En JavaScript, vérifier que :
 - Le nom n'est pas vide.
 - L'email contient un @.
3. Si le formulaire est valide :
 - Afficher un message de succès.
 - Sauvegarder le prénom dans LocalStorage.
4. Au rechargement de la page :
 - Vérifier si un prénom existe déjà dans LocalStorage.
 - Si oui → afficher : "Bienvenue, [Nom] 🖐️" en haut du CV.

Variante (bonus)

→ Ajouter une option "Se souvenir de moi" avec une case à cocher → si cochée, sauvegarder le nom dans LocalStorage, sinon l'ignorer.

LocalStorage : Mémoire du Navigateur

Le **LocalStorage** est une API web qui permet aux applications de stocker des données (paires clé/valeur) directement dans le navigateur de l'utilisateur, sans limite de temps. Idéal pour les préférences (thème sombre/clair), les noms d'utilisateurs ou les réglages personnalisés.

```
// Sauvegarder
localStorage.setItem("username", "Zineb");

// Lire
const user = localStorage.getItem("username");
console.log("Bienvenue " + user);

// Supprimer
localStorage.removeItem("username");
```

⚠️

Ne stockez **JAMAIS** de données sensibles (mots de passe, infos bancaires) dans le LocalStorage, car elles sont facilement accessibles.

Bases de Données & APIs

Jusqu'à maintenant, votre CV (ou votre site) contient uniquement du contenu **codé en dur** dans le HTML.

👉 Mais dans le monde réel, les sites modernes affichent des données **dynamiques** :

- Profil utilisateur,
- Produits d'un e-commerce,
- Derniers tweets ou posts,
- Articles d'un blog.

Ces données sont généralement stockées dans une **base de données** et accessibles via une **API**.

1. Qu'est-ce qu'une base de données ?

Une base de données est la **mémoire** d'un site web.

Elle permet de :

- Enregistrer des informations,
- Les organiser,
- Les retrouver facilement.

Deux grands types :

📊 SQL (relationnelles)

- Données organisées en **tables (lignes + colonnes)**.
- Chaque table a un schéma défini.
- Exemples : MySQL, PostgreSQL.

Exemple : **Table** `users`

id	name	email
1	Alice	alice@mail.com
2	Zineb	zineb@mail.com

```
CREATE TABLE users (id INT, name TEXT, email TEXT);
INSERT INTO users VALUES (1, "Alice", "alice@mail.com");
SELECT * FROM users;
```

📁 NoSQL (non relationnelles)

- Données sous forme de **documents (JSON)**.
- Flexible (pas besoin de schéma fixe).
- Exemple : MongoDB.

```
{ "id": 1, "name": "Alice", "email": "alice@mail.com" }
```

En résumé :

- SQL = structuré et strict.
- NoSQL = flexible et adaptable.

2. Qu'est-ce qu'une API ?

Une **API (Application Programming Interface)** est une **interface** qui permet à une application d'accéder à des données ou fonctionnalités sans se connecter directement à la base.

👉 Analogie :

Imaginez un **restaurant** :

- La base de données = la cuisine (où sont les ingrédients).
- L'API = le serveur qui prend la commande et apporte le plat.
- Le client = votre navigateur.

👉 Une API est donc un **intermédiaire** entre la base de données et l'utilisateur.

3. Les APIs REST et JSON

La plupart des APIs modernes sont :

- **REST** = architecture qui utilise les URLs pour accéder aux ressources.
- **JSON (JavaScript Object Notation)** = format de données léger et facile à lire.

Exemple : **API "utilisateurs"**

Appel de l'URL :

```
https://jsonplaceholder.typicode.com/users
```

Réponse JSON :

```
[
  { "id": 1, "name": "Leanne Graham", "email": "leanne@example.com" },
  { "id": 2, "name": "Ervin Howell", "email": "ervin@example.com" }
]
```

4. Consommer une API en JavaScript (Fetch)

JavaScript peut envoyer une requête HTTP et récupérer des données via la fonction `fetch()`.

Exemple simple

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then(response => response.json()) // convertir en JSON
  .then(data => console.log(data)); // afficher dans la console
```

👉 Ici, `data` est un tableau d'objets JSON représentant les utilisateurs.

Exemple avec affichage en HTML

```
<ul id="userList"></ul>

<script>
fetch("https://jsonplaceholder.typicode.com/users")
  .then(res => res.json())
  .then(users => {
    const list = document.getElementById("userList");
    users.forEach(user => {
      const li = document.createElement("li");
      li.innerText = `${user.name} (${user.email})`;
      list.appendChild(li);
    });
  });
</script>
```

👉 Résultat : une liste d'utilisateurs affichée automatiquement sur la page.

⚠️ Pièges fréquents

1. **Confondre API et base de données**
→ Une API ne *stocke* pas les données, elle les **expose**.
2. **Oublier de convertir en JSON** (`.json()`)
→ Sans ça, les données ne sont pas exploitables.
3. **Ne pas gérer les erreurs**
→ Exemple : l'API est hors ligne → il faut prévoir un message d'erreur.

💡 Bonnes pratiques

- Tester une API dans le navigateur avant de l'intégrer.
- Toujours vérifier que la donnée reçue existe (`if(user.name)`).
- Afficher un message "Chargement..." en attendant la réponse.

✍️ Exercice pratique (30 min)

Objectif

Apprendre à charger des données depuis une API et les afficher dans la page.

Étapes guidées

1. Créer une section "Utilisateurs" dans votre CV.

```
<h2>Utilisateurs</h2>
<table id="userTable">
  <tr><th>Nom</th><th>Email</th><th>Ville</th></tr>
</table>
```

1. Utiliser `fetch()` pour récupérer 5 utilisateurs depuis :
👉 `https://jsonplaceholder.typicode.com/users`
2. Pour chaque utilisateur, ajouter une ligne au tableau avec :
 - Nom
 - Email
 - Ville
3. Ajouter un bouton "Rafraîchir" qui recharge la liste.

Variante (bonus)

→ Ajouter une barre de recherche → afficher seulement les utilisateurs dont le nom contient le texte saisi.

Git et Gestion de Versions : Le Travail d'Équipe et la Sécurité

- 👉 "Comment travailler à plusieurs sans écraser le travail des autres ?"
- 👉 "Comment revenir en arrière si on casse quelque chose ?"

Lorsque vous développez, seul ou en équipe, la question de la gestion des modifications devient cruciale. **Git** est votre meilleur allié.

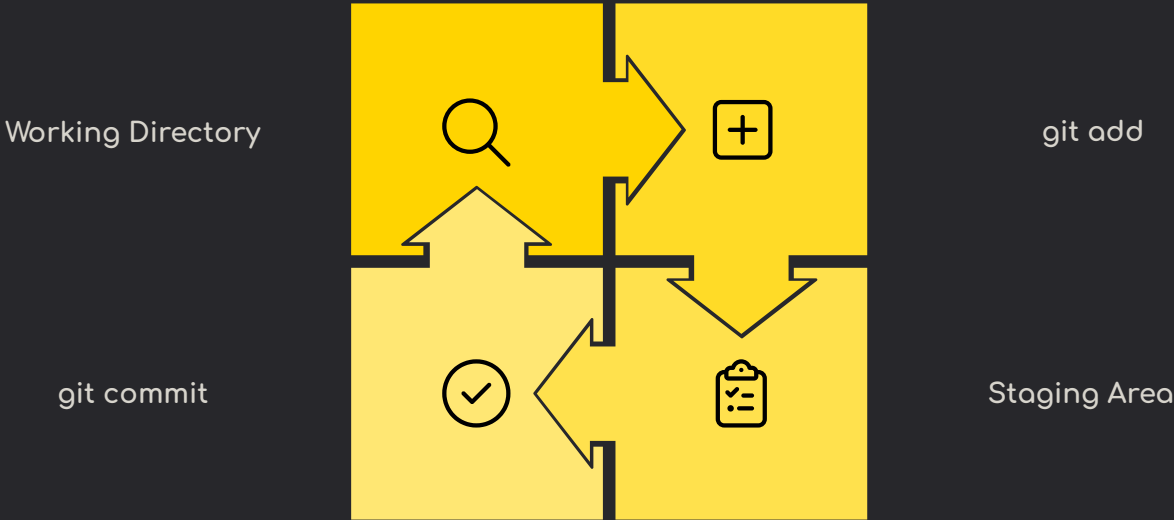
Sauvegarde Intelligente Chaque modification est enregistrée dans un historique détaillé, permettant de revenir à n'importe quelle version précédente.	Collaboration Facilitée Plusieurs développeurs peuvent travailler sur le même projet en parallèle sans écraser le travail des autres, grâce à des mécanismes de fusion.	Expérimentation Sécurisée Les "branches" permettent de développer de nouvelles fonctionnalités ou de tester des idées sans affecter la version principale stable du projet. C'est votre "machine à remonter le temps".
---	---	--

Imaginez que vous écrivez un **roman en groupe** :

- Sans Git : chacun écrit sur le même fichier Word → conflits, écrasements, chaos.
- Avec Git :
 - Chacun a une **copie complète** du livre.
 - On peut créer des **chapitres alternatifs** (branches).
 - Si une version est mauvaise, on peut revenir à une version antérieure.

👉 Git, c'est comme **un système de sauvegardes infinies + machine à remonter le temps**.

Les 3 Zones de Git



- Working Directory** : Vos fichiers locaux actuels, non suivis par Git.
- Staging Area** : Les modifications que vous avez sélectionnées avec `git add`, prêtes à être enregistrées.
- Repository** : L'historique des modifications validées via `git commit`.

Commandes Essentielles

```
git init /* Initialise un nouveau dépôt Git */
git status /* Affiche l'état des fichiers */
git add . /* Ajoute toutes les modifications à la Staging Area */
git commit -m "Message" /* Enregistre les modifications dans l'historique */
git log /* Affiche l'historique des commits */
git branch feature-x /* Crée une nouvelle branche */
git checkout feature-x /* Bascule sur une branche */
git merge feature-x /* Intègre les modifications d'une branche dans la branche actuelle */
```

⚠️ Pièges fréquents

- Tout travailler sur main**
→ Si on casse, on n'a plus de version stable.
✅ Solution : toujours créer des branches.
- Commits trop gros**
→ Un commit "modif tout" est inutilisable.
✅ Solution : faire des commits courts et descriptifs ("ajout grid layout CV").
- Oublier git add**
→ On croit avoir sauvegardé, mais rien n'a été validé.
✅ Solution : toujours vérifier avec `git status`.

💡 Bonnes pratiques en entreprise

- Convention de nommage des branches** :
 - feature/ajout-formulaire
 - fix/bug-responsive
- Convention de commits** :
 - "feat: ajout d'une navbar responsive"
 - "fix: correction bug formulaire email"
- Toujours travailler en équipe via **GitHub** ou **GitLab**.

📝 Exercice pratique (30 min)

Objectif

Mettre votre projet **CV du Jour 1/2** sous Git et apprendre à gérer des branches.

Étapes guidées

- Initialiser Git

```
git init
```

→ Créez un dépôt Git dans votre dossier CV.

- Premier commit

```
git add .
git commit -m "Version initiale du CV"
```

- Créer une **branche responsive**

```
git branch responsive
git checkout responsive
```

- Modifier votre CV**
→ Ajoutez un @media query pour la version mobile.
- Commit sur la branche responsive**

```
git add .
git commit -m "Ajout version responsive"
```

- Fusionner dans main**

```
git checkout main
git merge responsive
```


Mini-projet : Votre Première Landing Page Dynamique

Une **Landing Page** est une page d'accueil moderne conçue pour :

- **Attirer l'attention** (section Hero avec un slogan),
- **Présenter rapidement l'équipe / les services**,
- **Encourager une action** (formulaire de contact, bouton "S'inscrire").

👉 C'est un format très utilisé en entreprise (startups, produits SaaS, portfolios).

Aujourd'hui, vous allez créer une **Landing Page complète** qui combine **design responsive**, **interactivité** et **contenu dynamique**.

Cahier des Charges

- Votre landing page doit contenir :
 1. **Navbar responsive** (en Flexbox ou Grid)
 - Avec un logo fictif + 3 liens de navigation.
 - En version mobile, les liens passent en colonne.
 2. **Hero section**
 - Un titre accrocheur + un slogan.
 - Un bouton CTA ("En savoir plus").
 - Fond coloré ou image de fond.
 3. **Section Équipe (API)**
 - Charger des utilisateurs fictifs depuis l'API :
👉 `https://jsonplaceholder.typicode.com/users`
 - Afficher Nom + Email + Ville dans des cartes.
 4. **Formulaire de contact**
 - Champs : Nom, Email, Message.
 - Validation JS → pas vide, email valide.
 - Sauvegarde du prénom dans LocalStorage → message "Bienvenue [Nom]" affiché en haut de la page.
 5. **Footer**
 - Afficher l'année courante automatiquement en JS.
 - Bouton **Mode clair/sombre**.



Étapes Clés

Étapes guidées

Étape 1 : Structure HTML

- Créer les sections principales :

```
<header></header>
<section id="hero"></section>
<section id="team"></section>
<section id="contact"></section>
<footer></footer>
```

Étape 2 : Mise en page CSS

- Utiliser **Grid** pour la structure générale.
- Utiliser **Flexbox** pour aligner les éléments dans la navbar et la section équipe.
- Définir des **variables CSS** (`--primary-color`, `--text-color`).
- Ajouter 2 breakpoints :
 - Desktop (≥ 1024px) → Hero en 2 colonnes.
 - Mobile (≤ 768px) → Hero en 1 colonne.

Étape 3 : Interactivité avec JavaScript

- **DOM + événements** :
 - Bouton clair/sombre avec `.classList.toggle("dark-mode")`.
- **API (Fetch)** :
 - Charger et afficher 3 utilisateurs dans la section Équipe.
- **Formulaire + LocalStorage** :
 - Valider avant soumission.
 - Sauvegarder prénom dans LocalStorage → afficher un message personnalisé.
- **Footer dynamique** :

```
document.querySelector("footer").innerText = "© " + new Date().getFullYear();
```

⚠️ Pièges fréquents

1. **CSS trop rigide (px partout)**
→ Le site casse sur mobile.
✅ Solution : utiliser %, rem, vh/vw.
2. **Oublier preventDefault() dans le formulaire**
→ La page se recharge et efface les données.
3. **Ne pas tester le responsive**
→ Le site semble "bien" sur PC mais inutilisable sur téléphone.
✅ Solution : tester avec Chrome DevTools en mode mobile.

📝 Exercice (35 min en binômes)

Consigne

Chaque binôme doit réaliser la landing page avec :

- Navbar responsive,
- Hero avec CTA,
- Section Équipe alimentée par API,
- Formulaire validé + LocalStorage,
- Footer avec année dynamique + dark mode.

Variante (bonus pour les rapides)

- Ajouter une animation CSS (ex. fondu ou translation sur le Hero).
- Ajouter une barre de recherche dans la section Équipe → filtrer les utilisateurs